

CRUD App Build

Create, Read, Update, Delete -- The Core of Every App

■ LEARNING OUTCOME

By the end of this module you will build a complete full-stack CRUD application with a React frontend, Express backend, and database -- implementing all four operations with validation, error handling, and a polished UI.

■ Skills You Will Gain

- Design the data model and API contracts before coding
- Build a RESTful Express API with all CRUD endpoints
- Connect React frontend to backend with axios
- Implement client-side state management for CRUD operations
- Add form validation on both client and server
- Handle loading, error, and empty states professionally

■ Every web application -- social networks, e-commerce, project managers, content platforms -- is fundamentally a CRUD app. Master CRUD and you can build anything. It is the foundation skill of full-stack development.

SECTION 1

Data Model and API Design

```
# PROJECT: Task Manager (Todo App with extras) # Data model: Task # { # id: UUID primary key # title: string, required, max 200 chars # description: string, optional # status: 'todo' | 'in-progress' | 'done' # priority: 'low' | 'medium' | 'high' # due_date: date, optional # created_at: timestamp, auto # updated_at: timestamp, auto # user_id: FK to users (for auth later) # } # REST API DESIGN # Method Endpoint Action # GET /api/tasks List all tasks (with filter/sort/pagination) # GET /api/tasks/:id Get one task # POST /api/tasks Create new task # PUT /api/tasks/:id Full update (replace) # PATCH /api/tasks/:id Partial update (status change) # DELETE /api/tasks/:id Delete task # Query parameters: # GET /api/tasks?status=todo&priority=high&sort=due_date&order=asc&page=1&limit=20
```

SECTION 2

Express Backend

```
// backend/server.js const express = require('express'); const cors = require('cors'); const helmet = require('helmet'); require('dotenv').config(); const app = express(); app.use(helmet()); // security headers app.use(cors({ origin: process.env.CLIENT_URL })); app.use(express.json({ limit: '10kb' })); // parse JSON, limit body size app.use('/api/tasks', require('./routes/tasks')); app.use(require('./middleware/errorHandler')); app.listen(process.env.PORT || 3000); // backend/routes/tasks.js const router = require('express').Router(); const { body, param, query, validationResult } = require('express-validator'); const tasksCtrl = require('./controllers/tasksController'); // Validation rules const taskValidation = [ body('title').trim().notEmpty().isLength({ max: 200 }).withMessage('Title required, max 200 chars'), body('status').optional().isIn(['todo', 'in-progress', 'done']).withMessage('Invalid status value'), body('priority').optional().isIn(['low', 'medium', 'high']), body('due_date').optional().isISO8601().toDate().withMessage('due_date must be a valid date'), ]; const validate = (req, res, next) => { const errors = validationResult(req); if (!errors.isEmpty()) return res.status(422).json({ errors: errors.array() }); next(); }; router.get('/', tasksCtrl.getAll); router.get('/:id', tasksCtrl.getOne); router.post('/', taskValidation, validate, tasksCtrl.create); router.put('/:id', taskValidation, validate, tasksCtrl.update); router.patch('/:id', body('status').isIn(['todo', 'in-progress', 'done']), validate, tasksCtrl.patch); router.delete('/:id', tasksCtrl.remove); // backend/controllers/tasksController.js const db = require('./db'); // database module exports.getAll = async (req, res, next) => { try { const { status, priority, sort='created_at', order='desc', page=1, limit=20 } = req.query; const offset = (page - 1) * limit; const tasks = await db.task.findMany({ where: { ...(status && { status }), ...(priority && { priority }), }, orderBy: { [sort]: order }, skip: +offset, take: +limit, }); const total = await db.task.count(); res.json({ tasks, total, page: +page, pages: Math.ceil(total/limit) }); } catch (err) { next(err); } }; exports.create = async (req, res, next) => { try { const task = await db.task.create({ data: req.body }); res.status(201).json(task); } catch (err) { next(err); } }; exports.update = async (req, res, next) => { try { const task = await db.task.update({ where: { id: req.params.id }, data: req.body, }); res.json(task); } catch (err) { next(err); } }; exports.remove = async (req, res, next) => { try { await db.task.delete({ where: { id: req.params.id } }); res.status(204).end(); } catch (err) { next(err); } };
```

SECTION 3

React Frontend -- CRUD with State

```
// frontend/src/hooks/useTasks.js -- custom hook for task operations import { useState, useEffect, useCallback } from 'react'; import { tasksApi } from '../services/tasksApi'; export function useTasks() { const [tasks, setTasks] = useState([]); const [loading, setLoading] = useState(true); const [error, setError] = useState(null); const fetchTasks = useCallback(async (filters = {}) => {
```

```

setLoading(true); try { const { tasks } = await tasksApi.getAll(filters); setTasks(tasks); } catch
(e) { setError(e.message); } finally { setLoading(false); } }, []); useEffect(() => { fetchTasks();
}, [fetchTasks]); const createTask = async (data) => { const task = await tasksApi.create(data);
setTasks(prev => [task, ...prev]); // optimistic: add to top return task; }; const updateTask =
async (id, data) => { const updated = await tasksApi.update(id, data); setTasks(prev => prev.map(t
=> t.id === id ? updated : t)); return updated; }; const deleteTask = async (id) => { setTasks(prev
=> prev.filter(t => t.id !== id)); // optimistic remove try { await tasksApi.remove(id); } catch
(e) { fetchTasks(); // rollback on error throw e; } }; return { tasks, loading, error, createTask,
updateTask, deleteTask, refetch: fetchTasks }; } // frontend/src/components/TaskForm.jsx import {
useState } from 'react'; function TaskForm({ onSubmit, initial = {}, onCancel }) { const [form,
setForm] = useState({ title: '', description: '', priority: 'medium', due_date: '', ...initial });
const [errors, setErrors] = useState({}); const [loading, setLoading] = useState(false); const
validate = () => { const e = {}; if (!form.title.trim()) e.title = 'Title is required'; if
(form.title.length > 200) e.title = 'Title max 200 characters'; return e; }; const handleSubmit =
async (e) => { e.preventDefault(); const errs = validate(); if (Object.keys(errs).length) {
setErrors(errs); return; } setLoading(true); try { await onSubmit(form); } catch (err) {
setErrors({ submit: err.message }); } finally { setLoading(false); } }; return ( <form
onSubmit={handleSubmit} noValidate> <div className='form-group'> <label htmlFor='title'>Title
</label> <input id='title' type='text' value={form.title} onChange={e => setForm(p => ({...p,
title: e.target.value})} aria-invalid={!!errors.title} aria-describedby='title-error' />
{errors.title && <span id='title-error' role='alert'>{errors.title}</span> } </div> {errors.submit
&& <div role='alert' className='error'>{errors.submit}</div> } <div className='form-actions'>
<button type='submit' disabled={loading}> {loading ? 'Saving...' : initial.id ? 'Update Task' :
'Create Task'} </button> {onCancel && <button type='button' onClick={onCancel}>Cancel</button>
} </div> </form> ); }

```

■ PRACTICE Q & A

Q1. What is the difference between PUT and PATCH in a REST API?

A: *PUT: full replacement -- send entire updated resource. Server replaces all fields. PATCH: partial update -- send only fields to change. Server merges. Use PATCH for simple updates like changing status; PUT when replacing the whole resource.*

Q2. What is optimistic UI and when should you use it?

A: *Update the UI immediately before the server confirms, then roll back if the server returns an error. Makes the app feel instant. Use for low-risk operations like deleting a task or toggling status. Avoid for operations where failure is common.*

Q3. How should you handle form validation on both client and server?

A: *Client: validate immediately on submit (and optionally on blur) for fast feedback. Server: ALWAYS validate again independently -- client validation can be bypassed. Never trust client input. The server is the last line of defense.*

Q4. What HTTP status code should a successful POST return?

A: *201 Created. Include the created resource in the response body. 200 OK is for successful GET/PUT/PATCH. 204 No Content is for successful DELETE when you have nothing to return.*

Q5. What is the difference between a controller and a route in Express?

A: *Route: defines the URL pattern and HTTP method, applies middleware (validation, auth). Controller: contains the actual business logic for that route -- queries the database and formats the response. Separating them makes code testable and reusable.*

■■ CRUD App Cheat Sheet	
GET /api/tasks	List all (with filters)
POST /api/tasks	Create -- return 201 + created resource
PUT /api/tasks/:id	Full replace
PATCH /api/tasks/:id	Partial update
DELETE /api/tasks/:id	Delete -- return 204 No Content
express-validator	Server-side request validation
validationResult(req)	Check validation errors

<code>res.status(422).json({})</code>	Validation error response
<code>optimistic update</code>	Update state before server confirms
<code>try/catch with rollback</code>	Revert optimistic update on error
<code>useCallback for fetch</code>	Stable function ref for useEffect dep
<code>aria-invalid + role=alert</code>	Accessible form error announcement